8.9 最短路 / 最小生成树

主讲：Arkweedy

2025 年 8 月 8 日

最短路算法选择

1. 全源：Floyd
2. 单源：Dijkstra
3. 稀疏图：堆优化 Dijkstra
4. 稠密图：朴素 Dijkstra
5. **边权相同/为1/为01 ： bfs**

图论建模方法

1. 拆点，虚点(状态切换，特殊转移条件)
2. 分层图(状态切换，同余最短路)
3. 中心节点(点集内两两相连)
4. 超级源/汇点(多源点/汇点最短路问题)

最小生成树要点

1. 最小生成树—— 最大瓶颈路
2. 满足割性质
3. 代价最小使图联通
4. 贪心策略

- 课前准备：
  拥有Codeforces, Atcoder, Luogu账号
- 课上要求：
  集中精力，跟着思考，锻炼**思维**
- 课后要求：
  完成课后任务，巩固所学知识
- 课后任务的形式：
  完成题目，编写个人题解(学会使用Markdown / LaTeX)

回顾最短路算法:

- **单源非负权最短路：** Dijkstra (堆优化 $O(m \log n)$) 0-1 BFS (边权只有 0/1 时 $O(m+n)$)
- **单源允许负权最短路：** Bellman-Ford ($O(nm)$) SPFA (平均速度快，最坏情况退化为 $O(nm)$)
- **全源最短路：** Floyd-Warshall ($O(n^3)$) Johnson (稀疏图 $O(m \log n + n^2 \log n)$)

一句话总结

在选择最短路算法时，请先想清楚："有没有负权"、"是单源还是全源"以及"图是稠密还是稀疏"，再选择合适的算法。

松弛操作：所有最短路算法的灵魂

- **松弛操作 (Relaxation)** 是所有最短路算法的核心思想。
- 当我们发现一条边 (u, v) 使得从源点到 v 的距离可以通过 u 变得更短时，我们更新 $d[v]$ 的值。
- 形式化地，如果 $d[u] + w(u, v) < d[v]$，则更新 $d[v] = d[u] + w(u, v)$。
- 这一操作的目的是不断逼近真实的最短路。

问题

给定一张图，所有边的边权 $w \in (1, 2)$ 均为实数。请设计一个算法，在 $O(E)$ 的时间复杂度内求出最短路。

- 这个问题要求我们打破传统最短路算法的思维惯性，例如 Dijkstra 的 $O(E \log V)$ 或 Bellman-Ford 的 $O(VE)$。
- 它的解决方案正是"松弛"思想的经典极限应用。

- 这是一种巧妙的线性做法，通常称为 Scaling / 分层 0-1 BFS。
- **思路拆解：**
  1. **把"小数"变成"整数"：** 记 $d[u]$ 为源点到 u 的最短路，$\delta[u] = \lfloor d[u] \rfloor$ —— 只取整数部分。 **关键观察：** 由于每条边权 $w \in (1, 2)$，所以 $d[v]$ 与 $d[u]$ 的差值不超过 2。因此，$\delta[v]$ 至多比 $\delta[u]$ 大 1或2，不可能相等。这意味着 δ 值只会单调不减，且每次最多 +1或+2。这正好对应 0-1 BFS 的层数–分成3层!

- **思路拆解：**
  - **用多队列（桶）维护 δ：** 为了解决"同一桶（同一整层 δ）里的点相互不会再松弛"的问题，我们需要使用**3个队列**在模3的意义下进行操作。 3个队列可以确保在松弛过程中，同一"整数层"的节点不会相互影响，从而保持复杂度。 **核心思想：** 同一桶（同一整层 δ）里的点相互不会再松弛。原因：它们到源点的距离都在 $[k, k+1)$ 之间，而任何一条边再长也只有 < 2。所以，桶内任意两点 u, v，满足 $|d[u] - d[v]| < 1$，不可能再出现 $d[u] + w < d[v]$ 的新松弛。于是，每个顶点最多被推进桶一次、弹出一次，每条边最多检查一次——复杂度就天然是 $O(E)$。

Dijkstra 算法

- **贪心策略：** Dijkstra 算法是一种基于贪心思想的单源最短路算法，每次从**未访问**的顶点中选择距离源点最近的顶点，并将其加入已确定的最短路集合。
- **适用范围：** 仅适用于**边权非负**的图。
- **基本流程：** 不断选择当前距离最小的未访问顶点进行松弛操作。

Dijkstra 算法：堆优化 (稀疏图)

- **优化目的：** 解决朴素 Dijkstra 每次选择最小距离顶点耗时问题。
- **优化方法：** 使用**优先队列 (Priority Queue)** (通常为二叉堆) 维护未访问顶点及其当前最短距离。
- **时间复杂度：** $O(E \log V)$。
- **适用场景：** 边数 E 远小于 V^2 的稀疏图。

Dijkstra 算法：朴素实现 (稠密图)

- **实现方法：** 使用数组 ‘visited‘ 和遍历寻找最小距离的未访问顶点。
- **时间复杂度：** $O(V^2 + E)$。
- **适用场景：** 边数 E 接近 V^2 的稠密图，或顶点数 V 较小的情况。

Dijkstra 算法：实现选择策略

- **稀疏图 ($E \ll V^2$)：** 优先考虑**堆优化的 Dijkstra** ($O(E \log V)$)。
  边的数量相对较少，优先队列的优势能充分体现，每次提取最小距离顶点和更新距离的操作效率更高。
- **稠密图 ($E \approx V^2$)：** 可以选择**朴素的 Dijkstra** ($O(V^2 + E)$，近似 $O(V^2)$)。
  边的数量接近顶点数的平方，邻接矩阵表示图更方便，朴素实现中遍历所有顶点寻找最小距离的开销可以接受。
- **总结：** 在实际应用中，通常会根据图的稠密程度来选择合适的 Dijkstra 实现方式，以达到最优的时间效率。

例题：传送门 (经典拆点问题)

问题描述 (核心)

小S从1号遗迹到n号遗迹，需要支付最少金币。每个传送门属于一个魔法协会，连续使用同一协会传送门只支付一次金币，切换协会需重新支付。

- **难点：** 费用不仅取决于路径长度，还取决于**上一次使用的传送门协会**。这使得单纯的节点无法表示完整状态。
- **思考：** 如何将"当前所在遗迹"和"上一次使用的协会"这两个状态，融入到图的节点中？

拆点

- **核心思想：** 将每个原始遗迹节点 u 根据其进入的协会 c 进行"拆点"，形成新的节点 (u, c)。
- **新图节点定义:**
  - 原始遗迹节点 u $(1 \ldots n)$：表示小S当前在遗迹 u 但尚未决定使用哪个协会的传送门。
  - 拆分节点 (u, c)：表示小S当前在遗迹 u，并且上一次使用了协会 c 的传送门到达此地。

- **边权设定:**
  - 从原始遗迹节点 u 到拆分节点 (u, c)：边权为 1。表示小S从遗迹 u 开始，选择使用协会 c 的传送门，需要支付 1 枚金币。
  - 从拆分节点 (u, c) 到原始遗迹节点 v：边权为 0。表示小S从遗迹 u 通过协会 c 的传送门到达遗迹 v，如果继续使用协会 c，则无需额外支付。
  - 如果从 (u, c_1) 切换到 (v, c_2) $(c_1 \neq c_2)$：先从 (u, c_1) 到 u (边权0)，再从 u 到 (v, c_2) (边权1)。

- **具体建图:**
  - 为每个遗迹 u 和协会 c 的组合 (u, c) 创建一个唯一的"拆分节点"，分拆点向原点连边权为1的边。
  - 对于每个传送门 (u, v) 属于协会 c, 同色拆分点之间连边权为0的边。
- **最短路计算与结果:**
  - 从源点 '0' (对应遗迹 1) 开始跑 Dijkstra。
  - 最终结果：由于建图过程中每次金币支付会引入双向边导致路径长度翻倍，因此最终的最短路距离需除以 2 得到实际的金币数。

例题：奇偶最短路 (Codeforces Round 1025 D)

问题链接

https://codeforces.com/contest/1025/problem/D

核心思想

在某些图论问题中，我们需要在路径长度之外考虑额外的状态（例如，经过的边数奇偶性，或使用了特定数量的道具）。分层图是处理这类问题的强大工具，通过复制图的层来表示不同的状态。

- **问题特点：** 每次移动可以走任意 k 条边，且 k 是给定集合 A 中的一个元素。目标是判断从顶点 1 开始，能否到达每个顶点 i。
- **难点分析：** 关键在于 k 的奇偶性。一次移动 k 条边，会改

分层图

- **状态定义与分层：** 由于路径长度的奇偶性会影响后续的选择，我们将每个原始顶点 u 拆分为两个状态的节点：
  - 顶点 u 的"偶数层"节点 u_0：表示从源点到 u 的最短路径长度为偶数。
  - 顶点 u 的"奇数层"节点 u_1：表示从源点到 u 的最短路径长度为奇数。

- **建图：** 对于原始图中每条连接 u 和 v 的边：
  - 从 u_0 到 v_1 建立一条边，边权为 1。
  - 从 v_0 到 u_1 建立一条边，边权为 1。
  - 从 u_1 到 v_0 建立一条边，边权为 1。
  - 从 v_1 到 u_0 建立一条边，边权为 1。

  这样，每走一条边，都会从一个奇偶层切换到另一个奇偶层。

- **最短路算法：** 由于所有边权均为 1，我们可以使用 **BFS** 来求解从源点（通常是 1_0 或 1_1 视具体情况而定）到所有其他节点的最短路径长度。
- **结果处理：** 遍历给定的长度集合 A，计算其所有元素的总和 S。如果 S 是偶数，则可以到达所有偶数长度路径可达的节点；如果 S 是奇数，则可以到达所有奇数长度路径可达的节点。进一步地，我们可以通过动态规划（或观察性质）计算出所有可以达到的奇数/偶数长度总和，然后根据 BFS 得到的最短路径长度进行判断。

实际上,奇偶最短路可以扩展到模 m 的情况。

超级源/汇：概念与应用

- **概念：** 在某些需要计算多个源点到单个终点，或单个源点到多个终点的最短路问题中，我们常常会引入一个"虚拟"的超级源点 (Super Source) 或超级汇点 (Super Sink)。
- **超级源：** 当我们需要计算从多个起点到其他点的最短路时，可以新建一个超级源点 S，从 S 向所有真实起点连一条边权为 0 的边。然后以 S 为源点运行一次最短路算法，即可得到 S 到所有点的最短路，也就是所有真实起点到这些点的最短路的最小值。
- **超级汇：** 类似地，当我们需要计算从某个点到多个终点的最短路时，可以新建一个超级汇点 T，从所有真实终点向 T 连一条边权为 0 的边（如果图是无向的，或者需要反向建图）。
- **典型应用：**
  - 多源最短路问题。
  - 图中一个点集到另一个点集的最短路问题

例题：ChallengeNPCIII (2025浙江省赛F)

问题链接

(https://qoj.ac/contest/2021/problem/10728)

核心问题

给定一个有向图，每个顶点有颜色。判断对于任意长度为 k 的简单路径，路径上的顶点颜色是否都不同。

- **难点：** 需要检查所有可能的简单路径，且路径长度为 k。直接遍历所有路径是不现实的。
- **转化思路：** 问题的等价表述是：是否存在一条长度为 k 的简单路径，其中包含重复颜色。我们通常会尝试寻找反例。

超级源/汇

- **问题转化：** 题目要求"每条长度为 k 的简单路径包含不同颜色"。其反面是"存在一条长度为 k 的简单路径包含重复颜色"。
- 我们可以逐个颜色进行检查：对于每种颜色 C，是否存在从某个颜色为 C 的顶点 u 到另一个颜色为 C 的顶点 v 的路径，这条路径（不含 u, v）的长度恰好是 $k-2$，并且路径中间不经过颜色为 C 的顶点。
- 如果存在这样的路径，那么这条路径加上 u 和 v 两个端点，就形成了一条长度为 k 的简单路径，且包含了重复颜色 C。

- **转化为最短路问题：** 对于每种颜色 C，构建一个子图，其中只包含颜色不为 C 的顶点和相关的边。然后在这个子图中，我们需要计算从所有颜色为 C 的顶点到所有其他颜色为 C 的顶点之间的最短路。
- 如果某个最短路为 L，则原图中的路径长度为 $L+2$（加上

超级源/汇

对于每种颜色 C，直接对所有同色顶点对跑最短路是低效的。我们可以利用超级源/汇和二进制分组来优化这个过程。
遍历每种颜色 C，对于颜色 C 的所有顶点集合 P_C：
引入一个超级源点 S 和一个超级汇点 T。将 P_C 中的顶点进行二进制分组。对于每个二进制位 b 和其取值 0 或 1：构建一个辅助图：只保留原图中颜色不为 C 的顶点和边。将 P_C 中第 b 位为 0 的顶点与 S 相连（边权0），将第 b 位为 1 的顶点与 T 相连（边权0）。（注意：这里的连接方式需要根据具体的二分匹配思想来调整，以实现多源多汇的最短路计算）

在辅助图上跑一次 BFS（或 Dijkstra，因为边权为1），计算 S 到 T 的最短路。通过二进制分组的性质，我们可以判断是否存在某个 $u \in P_C$ 到 $v \in P_C$ 的最短路径长度恰好为 $k-2$。
如果找到了任何一种颜色 C 存在这样的路径，则输出"NO"；否则输出"YES"
时间复杂度： 对于每种颜色，进行 $\log |P_C|$ 次 BFS。总复杂度为 $\sum_C O((N+M)\log |P_C|)$，在所有颜色总点数和总边数限制下，可以达到 $O((N+M)\log N)$。

最小生成树 (MST) 一句话总结

核心要点

解决 MST 的问题时，首先要理解 **Cut Property (割性质)**，然后选择 **Kruskal** 或 **Prim** 算法。

Kruskal 算法

- **核心思想：** 贪心地选择图中权值最小的边，前提是这条边连接了两个不同的连通分量，并且不形成环。
- **流程：**
  1. 将所有边按权值从小到大排序。
  2. 维护一个并查集，初始时每个顶点都在独立的集合中。
  3. 依次遍历排序后的边，如果当前边的两个端点不在同一个集合中，则选择这条边（加入到MST中），并合并这两个端点所在的集合。
  4. 重复直到选择了 $V-1$ 条边，或者所有边都被检查完。
- **时间复杂度：** $O(E \log E)$ (主要在排序) 或 $O(E \log V)$ (若使用优化的并查集)。
- **适用场景：** 稀疏图。

Prim 算法

- **核心思想：** 从一个起始顶点开始，逐步向外扩展，每次选择连接当前已构建的 MST 与未加入 MST 的顶点之间权值最小的边，将其顶点加入到 MST 中。
- **流程：**
  1. 随机选择一个起始顶点，将其加入到 MST 顶点集合 S 中。
  2. 维护一个优先队列，存储所有连接 S 中顶点与 $V - S$ 中顶点的边。
  3. 重复 $V - 1$ 次：
     - 从优先队列中取出权值最小的边 (u, v)，其中 $u \in S, v \in V - S$。
     - 将 v 加入到 S 中，并将边 (u, v) 加入到 MST 中。
     - 更新所有与 v 相邻的未在 S 中的顶点到 S 的最短距离。
- **时间复杂度：** 朴素实现 $O(V^2)$；堆优化 (使用优先队列) $O(E \log V)$。
  适用场景：稠密图 (朴素实现)，稀疏图 (堆优化)

Cut Property（割性质）

定义

对图的任意一个割 $(S, V-S)$，跨越该割且权值最小的那条边一定存在于至少一棵最小生成树中。

- **割 (Cut)**：将图的顶点集合 V 分成两个非空、不相交的子集 S 和 $V-S$。
- **跨越割的边 (Crossing Edge)**：连接 S 中顶点和 $V-S$ 中顶点的边。
- **重要性：** 这是所有 MST 算法正确性的基石。它保证了我们每次贪心地选择最小权值的跨割边，最终能够构建出最小生成树。
- **核心理解：** 理解"任意割都有一条必须选的最轻边"，从本质上理解 MST 的贪心策略。

Kruskal 算法与 Cut Property

- **Kruskal 视角：** Kruskal 算法是"全局排序后依次砍"边的过程。
- **应用割性质：** 当 Kruskal 算法按边权从小到大遍历边时，一旦遇到一条连接两个不同连通分量的边，它就立即选择这条边。
- **本质：** 这条被选择的边，实际上就是连接这两个连通分量所形成的"割"的最小跨越边。
- **理解：** Kruskal 算法的正确性正是因为每一步都遵循了 Cut Property，保证了所选择的边是最优的，不会错过任何一棵 MST 中的关键边。

Prim 算法与 Cut Property

- **Prim 视角：** Prim 算法是"从 S 开始局部扩张"的过程。
- **应用割性质：** Prim 算法每次从当前已构建的 MST 顶点集合 S（作为割的一侧）与其余顶点 $V - S$ 之间，选择权值最小的跨越边，并将其对应的顶点加入到 S 中。
- **本质：** Prim 算法每次选择的边，正是连接当前割 $(S, V - S)$ 的最小跨越边。
- **理解：** Prim 算法的正确性同样来源于 Cut Property。它保证了每一步的局部最优选择最终会导向全局最优的最小生成树。

问题链接

https://codeforces.com/gym/105385/problem/J

核心问题

给定 n 种颜色，每种颜色有 $-i$ 个顶点。任意两个顶点之间都有边，边权仅与两端点的颜色有关，为 b_{c_u,c_v}。求整个图的最小生成树总权值。

- **图的特点：** 这是一个完全图，顶点数量可能非常大 ($\sum -i$)，但不同颜色数量 n 相对较小 ($n \leq 1000$)。
- **难点：** 顶点数量太大，无法直接对所有顶点运行 Kruskal 或 Prim 算法。
- **转化思路：** 观察到边权只与颜色有关。这提示我们可以将问题转换为对"颜色"而不是"顶点"进行操作。

- **核心思想：** 尽管顶点数量巨大，但图中边的权值只与顶点的颜色有关。这意味着我们可以将问题抽象为在"颜色"之间构建最小生成树，而不是在具体的顶点之间。
- 我们可以把每种颜色视为一个"超级节点"，然后考虑如何连接这些超级节点，以及如何在每个超级节点内部进行连接。
- **DSU 的作用：** 我们可以使用并查集 (Disjoint Set Union, DSU) 来维护颜色之间的连通性。每个集合代表一个已连接的颜色组。
- 初始时，每种颜色（每个超级节点）都是独立的连通分量。

Kruskal 算法的视角： 我们可以将所有颜色对之间的边权 $b_{i,j}$ (包括 $b_{i,i}$) 提取出来，构成一个新的"颜色图"的边集，然后对这些边进行排序。

边的分类与贡献：在处理这些边时，我们需要区分两种情况：

同色顶点之间的边 ($i = j$)：

- 权值为 $b_{i,i}$。对于颜色 i 的 $-i$ 个顶点，为了使它们连通，最少需要 $-i-1$ 条边。这些边都连接同种颜色的顶点，权值都是 $b_{i,i}$。
- 只有当 $-i > 1$ 时，才需要考虑在颜色 i 内部连接顶点。如果 $-i = 1$，则无需内部连接。
- 贡献：如果某个连通分量只包含颜色 i 的顶点，且数量大于 1，则可以使用 $-i-1$ 条权值为 $b_{i,i}$ 的边来连接它们。这部分贡献需要在 Kruskal 算法开始前或过程中特殊处理。

不同颜色顶点之间的边 ($i \neq j$)：

- 权值为 $b_{i,j}$。这些边连接的是不同颜色组的顶点。
- Kruskal 算法会优先选择权值小的 $b_{i,j}$ 来连接不同的颜色连通分量。
- 贡献：当选择一条连接颜色 i 和颜色 j 的边时，它会使颜色 i 和颜色 j 所在的连通分量合并。这条边的贡献是连接这两个连通分量所需的最小代价，由当前 $b_{i,j}$ 决定。

- **总权值的累加逻辑：**
  1. **内部连接：** 对于每种颜色 i，如果其顶点数量 $-i > 1$，我们始终可以花费 $b_{i,i}$ 的权值来连接其中 $-i-1$ 个顶点，使它们内部连通。这部分权值在 Kruskal 排序中是优先级最高的同色边。
  2. **连通不同颜色组：** 对所有 (i,j) 对的边权 $b_{i,j}$ (包括 $i=j$ 的情况) 进行排序，然后用 Kruskal 的思想处理。
     - 当选择连接颜色 i 和颜色 j 的边时 (即通过 $b_{i,j}$ 连接两个连通分量)：
     - 如果 $i=j$，且这种颜色的点仍未完全连通，那么会增加 $(-i-1)$ 条 $b_{i,i}$ 权值的边，使得该颜色内部的点完全连通。
     - 如果 $i \neq j$，且颜色 i 和颜色 j 所在的连通分量目前不连通，那么选择这条边将合并这两个连通分量。此时，我们连接了 $-i$ 个颜色为 i 的顶点和 a_j 个颜色为 j 的顶点，需要花费 $b_{i,j}$。这相当于连接了所有 $-i+a_j$ 个点。而 Kruskal 算法每次连接两个连通分量时，会消耗一个"连通代价"，这里是 $b_{i,j}$。

- **特殊处理：** Kruskal 算法通常是选择 $V-1$ 条边。在这个抽象的"颜色图"中，我们有 n 个"超级节点"。因此，我们需要选择 $n-1$ 条边来连接所有颜色。
  这些连接颜色组的边，其权值由 $b_{i,j}$ 决定。

- **最后一步：** 在所有颜色都通过 $n-1$ 条边（代表着异色连接）连通之后，对于每个颜色 i 内部尚未连通的顶点，都需要用 $b_{i,i}$ 的权值来将它们连通。但是，由于 Kruskal 算法本身会处理同色边的连通，这里只需要考虑每种颜色内部剩余未连接的点。如果某种颜色的点多于1个，且在合并过程中并未作为"桥梁"连接到其他颜色，那么这部分点需要通过自身内部的最小代价边 $b_{i,i}$ 来连通。
  这个过程会在 Kruskal 算法中自然发生，通过维护每个颜色组内已连接点的数量来判断。

场景回顾：瓶颈路

- **瓶颈路：** 指的是连接图中两点所有路径中，最大边权值最小的那条路径。
- **挑战：** 如何高效地查询任意点对之间的瓶颈路权值？
  - **暴力：** 枚举所有路径，时间复杂度爆炸。
  - **普通 MST：** 在最小生成树上，两点之间的路径是唯一的。瓶颈路权值就是这条唯一路径上的最大边权。每次查询需要 $O(\log N)$（使用 LCA 和路径最大值数据结构）。
  - **局限性：** 如果查询次数 $Q \approx N^2$，则总复杂度为 $O(N^2 \log N)$，无法接受。
- **Kruskal 重构树的引入：** KRT 旨在将瓶颈路问题彻底离线化。
  - **优势：** 一次性 $O(M \log M)$ 建树，之后任意点对瓶颈路查询可达 $O(1)$ 或 $O(\log N)$ 且常数极小。

Kruskal 重构树 (KRT)

- **输入：** 无向图 $G = (V, E, w)$，其中 V 是顶点集， E 是边集， w 是边权。
- **输出：** 一棵二叉树 T，其叶子节点对应原图的 N 个顶点，内部节点代表连接两个连通块时的那条边。
- **算法流程 (与 Kruskal 算法同步进行)：**
  1. 将所有边 (u, v, w) 按权值 w 升序排序。
  2. 初始化并查集：每个原图节点 i $(1 \ldots N)$ 都是一个独立的集合。
  3. **关键：** 除了原图的 N 个叶子节点，我们还会创建额外的内部节点。初始时，每个叶子节点代表一个连通块，其编号为 $1 \ldots N$。

Kruskal 重构树 (KRT)

- **算法流程 (续)：**
- 扫描每条排序后的边 (u, v, w)：
  - 如果 u, v 不在同一个集合中 (即它们的根不同)：
    - **新建内部节点：** 创建一个新的内部节点 p，其权值 $val[p] = w$。这个节点将代表连接 u 和 v 所在连通块的这条边。
    - **建立父子关系：** 将 u 所在树的根节点和 v 所在树的根节点分别作为新节点 p 的左右儿子。
    - **合并集合：** 在并查集中，合并 u 和 v 所在的集合，并将新节点 p 作为合并后的新集合的根。原先的根节点现在成为 p 的子节点。
- 重复上述过程，直到所有顶点都被连接到一个大的连通分量中 (即并查集中只剩下一个集合)。
- **最终结果：** 算法结束后会得到一棵包含 $2N - 1$ 个节点的二叉树。其中 N 个叶子节点对应原图的 N 个顶点，另外 $N - 1$ 个内部节点对应 Kruskal 算法中选取的 $N - 1$ 条边。

Kruskal 重构树的性质

- **叶子节点：** 对应原图的 N 个顶点。
- **内部节点：** 每个内部节点代表一条在 Kruskal 算法中被选取的边，其权值就是该边的权值。
- **核心性质：** 任意原图中的两个点 u, v，它们在 Kruskal 重构树上最近公共祖先 (LCA) 的权值 (即内部节点的权值) 就是原图中 u 到 v 的所有路径中，最大边权值的最小值 (即瓶颈路权值)。
- **理由：** $LCA(u, v)$ 对应的边是第一次将 u 和 v 所在的连通块连接在一起的那条边。这意味着从 u 到 v 的所有路径中，都必须经过这条边，因此它是路径上的最大边权。
- 此外，由于 Kruskal 算法是按边权升序选择的，所以这条边是连接 u 和 v 的所有可能路径中，最大边权值最小的那个。

Kruskal 重构树：查询与复杂度

- 查询任意两点 u, v 的瓶颈路权值，只需计算它们在 Kruskal 重构树上的最近公共祖先 (LCA) 的权值。
- 为了高效地查询 LCA，我们需要对 Kruskal 重构树进行预处理：
  - **Euler Tour + ST 表：** 可以在 $O(N \log N)$ 时间内预处理，实现 $O(1)$ 的 LCA 查询。常数极小，非常适合大量查询。
  - **树链剖分：** 也可以在 $O(N)$ 时间内预处理，实现 $O(\log N)$ 的 LCA 查询。在竞赛中也常用。

复杂度分析：

- **建树：** $O(M \log M)$ (与 Kruskal 算法相同，主要时间消耗在边的排序和并查集操作)。
- **LCA 预处理：** $O(N \log N)$。
- **单次查询：** $O(1)$ (使用 ST 表) 或 $O(\log N)$ (使用倍增)。这里的 N 是重构树的节点数，即 $2N_{original} - 1$。
- **优势：** 相较于在普通 MST 上每次查询 LCA，KRT 的查询常数通常更小，特别适合需要频繁查询瓶颈路权值的场景。

例题：G. Omg Graph

问题链接

https://codeforces.com/contest/2117/problem/G

问题描述 (核心摘要)

给定一个无向连通带权图。定义一条路径的代价为：路径上所有边的最小权值 ($\min w_i$) 加上最大权值 ($\max w_i$)。在所有从顶点 1 到 n 的路径中，找到代价最小的路径，并报告这个最小代价。路径不一定是简单路径。

"Omg Graph"问题：核心思路 (1/3) – 问题转化

- **特点：** 路径代价由路径上的**最小边权** ($\min w_i$) 加上**最大边权** ($\max w_i$) 决定，目标是最小化这个和。
- **难点：** 同时考虑路径上的极值，且路径可非简单。

最终例题：G. Omg Graph (Codeforces Round 1029 Div. 3 G)

- **子问题：** 在固定了 $W_{\max}$ 后，我们只考虑权值 $\leq W_{\max}$ 的边构成的子图。在这个子图中，我们需要找到一条从 1 到 n 的路径，使得路径上的**最小边权 $W_{\min}$ 尽可能大**。
- **为什么寻找最大的 $W_{\min}$？** 因为我们最终要最小化 $W_{\min} + W_{\max}$。当 $W_{\max}$ 固定时，如果 1 和 n 能连通，为了使 $W_{\min} + W_{\max}$ 最小，我们实际上需要 $W_{\min}$ 尽可能大，这样才能覆盖到更多的、更小的 $W_{\max}$ 的情况。
- **最终目标：** 遍历所有可能的 $W_{\max}$，对于每个 $W_{\max}$，在相应的子图中找到连接 1 和 n 的路径中最大的 $W_{\min}$，然后取所有 $(W_{\min} + W_{\max})$ 中的最小值。

最终例题：G. Omg Graph (Codeforces Round 1029 Div. 3 G)

- **算法思路：** 我们可以借鉴 Kruskal 算法的思路。将图中的所有边按照权值从小到大排序。
- **遍历边与维护：** 依次将排序后的边加入图中。在加入边的同时，我们需要动态维护连通性，并更新每个连通分量内已包含的最小边权和最大边权。
- **并查集 (DSU) 的扩展：** 使用并查集来维护图的连通分量。对传统的 DSU 进行扩展：
  - 每个连通分量的根节点除了维护其父节点和集合大小外，还需要额外存储两个信息：当前连通分量中所有边的 **最小权值** e_{min} 和 **最大权值** e_{max}。
  - 初始时，每个顶点自成一个连通分量，其 e_{min} 初始化为一个很大的值 (例如无穷大)，e_{max} 初始化为一个很小的值 (例如 0)。
  - 当合并两个连通分量 ‘C1‘ 和 ‘C2‘ (通过一条权值为 ‘w‘ 的边) 时：变化是什么？

最终例题：G. Omg Graph (Codeforces Round 1029 Div. 3 G)

- **路径代价计算：** 在按权值从小到大遍历边的过程中，一旦源点 1 和目标点 n 通过当前加入的边以及之前加入的边变得连通时：
  - 这意味着存在一条或多条连接 1 和 n 的路径。
  - 并且，这些路径上的所有边权都小于或等于当前正在处理的边的权值 (因为边是按升序加入的)。
  - 此时，当前 1 和 n 所在连通分量的根节点上维护的 e_{minin} 和 e_{max} 就包含了从 1 到 n 的某条路径上的最小和最大边权。
  - 它们的和 $(e_{minin} + e_{max})$ 就是一个**候选的最小代价**。我们用这个值来更新全局的最小答案。
- **正确性：** 这种方法能够找到全局最小代价路径，是因为我们遍历了所有可能的最大边权 $W_{\max}$ (即每次连通 1 和 n 时的当前边权 ‘w‘)，并对于每个 $W_{\max}$，找到了在该限制下连接 1 和 n 的路径中最大的 $W_{\min}$。

最终例题：G. Omg Graph (Codeforces Round 1029 Div. 3 G)

思考：如果在$1, n$连通时，计算$e_{minin} + e_{max}$， break出去，答案是否正确？

"Omg Graph"问题：加强版 – 多点询问

加强版问题描述

原题只要求从顶点 1 到 n 的最小代价路径。加强版则可能需要**回答多次询问**，每次询问给出两个任意顶点 u, v，要求报告从 u 到 v 的最小代价路径。

- **挑战：** 如果每次询问都重新运行上述 Kruskal 变种算法，效率会非常低下 ($O(Q \cdot M \log M)$)。我们需要一种能够高效处理多点对查询的方法。
- **思路：** 联想到之前学习的 Kruskal 重构树 (KRT)，它能够高效地查询任意两点之间的瓶颈路权值。这为我们解决加强版问题提供了方向。

最终例题：G. Omg Graph (Codeforces Round 1029 Div. 3 G)

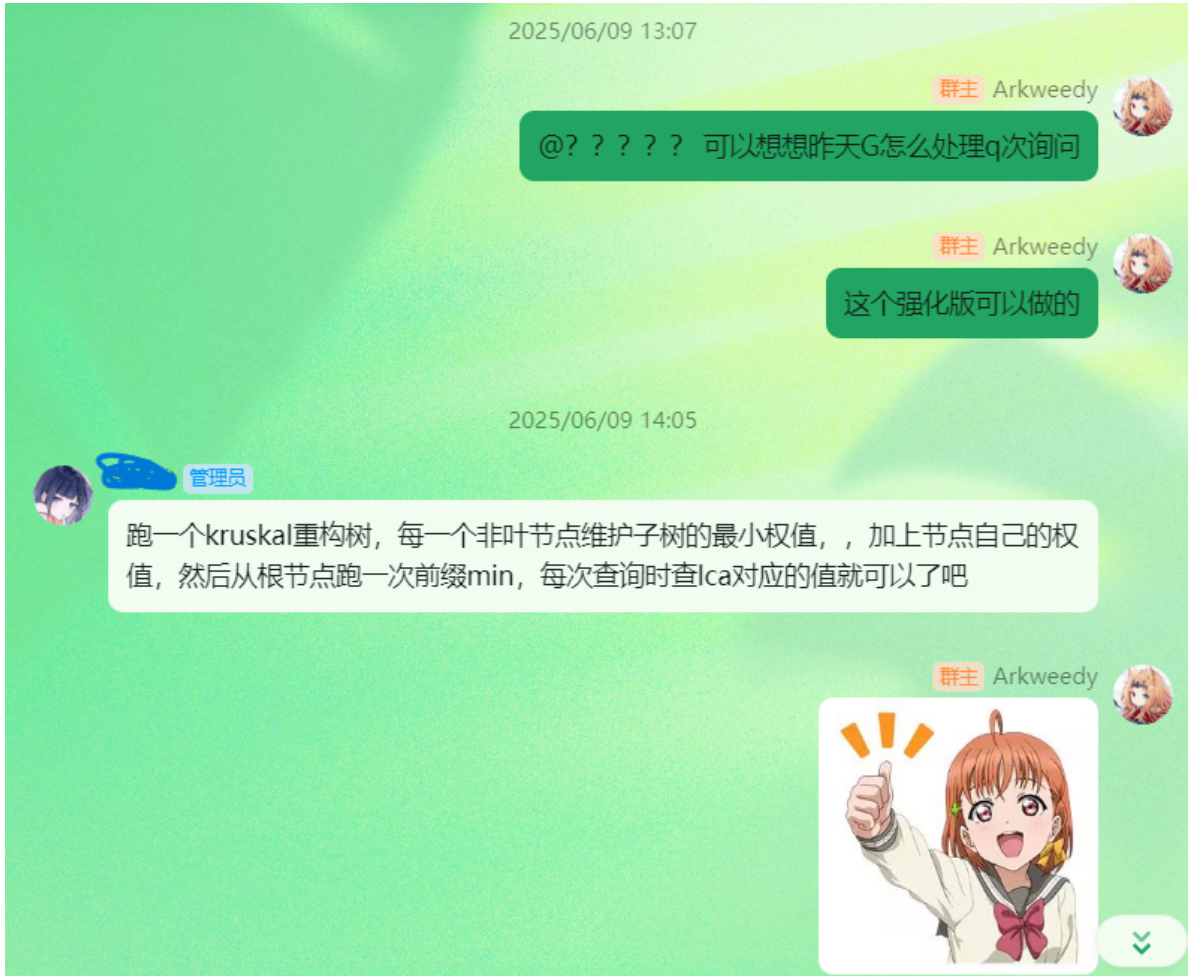

图: 学弟太强了

最终例题：G. Omg Graph (Codeforces Round 1029 Div. 3 G)

- **Kruskal 重构树的构建：** 依然按照 Kruskal 算法的流程，将所有边按权值从小到大排序，并使用并查集。
- **内部节点的额外信息：** 在构建 Kruskal 重构树时，每个新创建的内部节点 p (它代表一条连接两个连通分量权值为 w 的边) 不仅存储 w 作为其权值 $val[p]$，还需要额外存储其子树中所有叶子节点 (即原图顶点) 对应的路径的**最小边权**和**最大边权**。
  但这里我们维护的是瓶颈路问题，所以内部节点本身就代表了一个最大边权。
- **叶子节点的属性：** 原图的 N 个顶点 (叶子节点) 在重构树中，可以认为其自身的权值为 0，或者说其代表的路径只包含自身，没有边。
- **树的形态：** 最终形成一棵二叉树，叶子节点是原图顶点，内部节点是合并操作产生的，其权值为对应边的权值。

最终例题：G. Omg Graph (Codeforces Round 1029 Div. 3 G)

- **查询任意点对 (u, v)：** 对于任意一对查询点 u, v，我们需要在 KRT 上找到它们的最近公共祖先 $LCA(u, v)$。
- **KRT 节点的意义：** $LCA(u, v)$ 节点在 Kruskal 重构树中，其权值 $val[LCA(u, v)]$ 即为连接 u 和 v 的所有路径中，最大边权值的最小值 (即瓶颈路权值)。这个值，就是我们要求的 $W_{\max}$。
- **寻找 $W_{\min}$：** 确定了 $W_{\max}$ 后，我们需要找到在所有边权 $\leq W_{\max}$ 的路径中，连接 u, v 的最小边权 $W_{\min}$。
- **树上前缀最小：** 可以在 KRT 的每个节点上，额外维护从根节点到当前节点路径上的最小边权 (这里的"边"是指在KRT中连接父子节点的边，其权值就是KRT内部节点的val)。
- **优化查询：** 通过在 KRT 上进行 LCA 查询，并结合树上前缀最小 (或通过对子树的预处理)，我们可以在 $O(\log N)$ 甚至 $O(1)$ 的时间内完成单次查询。

整体流程：

1. 构造 Kruskal 重构树：$O(M \log M)$。
2. 对 KRT 进行预处理，支持 LCA 查询和树上路径最小/最大值查询：$O(N \log N)$。
3. 每次查询 (u, v)：
   - 找到 $LCA(u, v)$。这个节点的权值就是 $W_{\max}$。
   - 在 KRT 上，找到 u 到 $LCA(u, v)$ 的路径上的最小边权，以及 v 到 $LCA(u, v)$ 的路径上的最小边权。这两个值的最小值，就是我们需要的 $W_{\min}$。
   - 最终答案为 $W_{\min} + W_{\max}$。